Learning from Big Data: Assignment 2

Recommending fashion items with Thompson Sampling and UCB

Dongjie (Leo) Wu Qiu, 721207 STUDENTNUMBER

October 13, 2026

1 Introduction

ZOZOTOWN, a large Japanese fashion website, shows every visitor a row of three recommended items. Which items should go into that row, and in which slot, to get the most clicks? With dozens of candidate items and click rates well below one percent, the site cannot simply test every item at length: every impression spent on a weak item is a click lost. This is a multi-armed bandit problem. Each item is an arm, a click is the reward, and the policy has to balance trying uncertain items (exploration) against showing the items that look best so far (exploitation).

We use a week of logged data from a randomised experiment that ZOZO ran in November 2019 (?). We implement Thompson Sampling (TS) and the Upper Confidence Bound algorithm (UCB) and evaluate them offline with the replay method of ?. Beyond the comparison asked for in question (a), we study batch updates (b), tuning (c), user heterogeneity (d), fairness (e), stationarity (f) and policies for a changing environment (g). Two extensions are implemented: a minimum exposure guarantee for items, and discounted TS and sliding window UCB for non-stationary data.

[TODO: Two or three sentences with the headline findings, written last.]

2 Data

The main file has 1,356,670 impressions of 79 items over seven days (24 to 30 November 2019), with 4,720 clicks, an overall click-through rate (CTR) of 0.348%. Each row records one item shown in one slot (1 = left, 2 = centre, 3 = right), whether it was clicked, its propensity score and four hashed user features. Two smaller files contain the same log restricted to 10 items (172,400 rows, CTR 0.280%) and 20 items (345,469 rows, CTR 0.255%); the 10 item file has no user features. We use the 80 item file for all main results and the 10 item file as an easier setting with fewer arms (code Table 1).

Three facts about the data drive the rest of the analysis.

The logging policy was uniform random. Every row has propensity $0:0125 = 1=80$ (code Table 2). The brief mentions that ZOZO also ran a Bernoulli TS policy, but those impressions are not in this file, so the policy that generated our data, and our benchmark, is uniform random serving. Uniform logging is also what makes unbiased replay evaluation possible (Section ??).

Items differ, positions hardly do. Item CTRs range from 0.115% to 0.784% (Figure ??), so the best item is clicked almost seven times as often as the worst. That spread is what a bandit can exploit. The three positions have almost the same CTR: 0.354%, 0.349% and 0.341% (code Table 3).

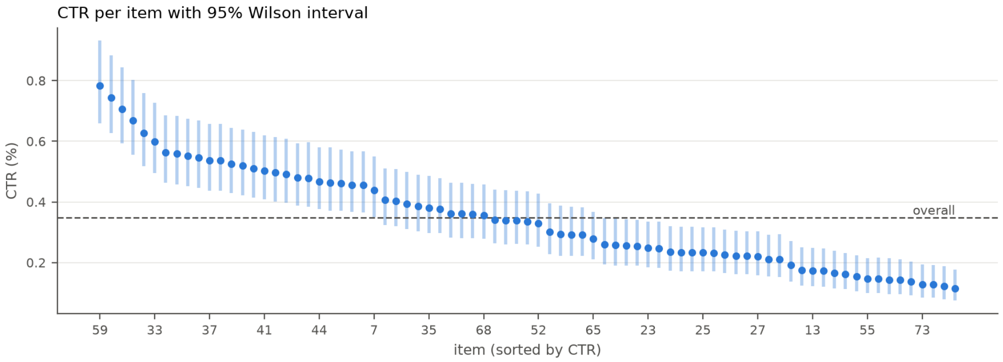

Figure 1: CTR per item in the 80 item file with 95% Wilson intervals, sorted from best to worst; the dashed line is the overall CTR (code Figure 1).

One user code covers a third of the data. In all four user features the same code covers exactly 479,757 rows (35%), which suggests these are visitors without a known profile (code Table 5). Several feature levels are tiny (some have fewer than 100 rows). This matters for questions (d) and (e).

Daily CTR moves between 0.303% and 0.382% (code Table 6, Figure 2); we test in Section ?? whether this is more than noise.

3 Method

3.1 Offline evaluation by replay

We cannot show items to real visitors, so every policy is evaluated on the log with the replay method (?). Let the log be a sequence of events (x_t, a_t, r_t), $t = 1, \ldots, T$, in time order, where x_t is the context (slot and user features), a_t the item that was shown and $r_t \in \{0, 1\}$ the click. A policy π walks through the log. At each event it picks an item $\pi(x_t, h_{t-1})$ based on its history h_{t-1}. If the pick equals the logged item, the event is matched: the click is revealed, counted, and added to the history. Otherwise the event is skipped and the policy learns nothing. The replay estimate of the policy's CTR is

$$\widehat{\mathrm{CTR}}(\pi) = \frac{\sum_{t=1}^{T} r_t \mathbb{1}\{\pi(x_t; h_{t-1}) = a_t\}}{\sum_{t=1}^{T} \mathbb{1}\{\pi(x_t; h_{t-1}) = a_t\}}. \qquad (1)$$

? show that this estimate is unbiased when the logged item was chosen uniformly at random and independently of the click, and the events are independent. The first condition holds by design. With K arms, about one event in K matches, so both the 10 and the 80 item file leave roughly 17,000 matched events and only 45 to 130 clicks per run. One standard error of a CTR estimate is 0.04 to 0.07 percentage points, and we treat differences of that size as noise.

The independence condition holds only approximately. In the log, the same item often stays in the same slot for many consecutive rows. Deterministic policies such as UCB keep the same pick until they learn something new, so they match one row of such a streak and then wait for the next streak of their new favourite: UCB with $c = 1$ matches about 5,000 events in time order but about 17,000 when the rows are shuffled (code Table 18). We report matched events next to every CTR and return to this in Section ??.

Each stochastic policy is replayed 10 times (5 times in the larger grids), run i using random seed $1 + i$. Tables report the mean CTR over runs, the spread across runs, and the binomial standard error $\sqrt{\hat{p}(1-\hat{p})/n}$ of one run with n matched events. The benchmark at the top is the Oracle, which always shows the item with the highest CTR over the whole week. It uses information no real policy has, so it is an upper bound for any policy that shows the same item to everyone.

3.2 Thompson Sampling

TS keeps a Beta belief about the click probability $_k$ of every item k. Starting from the prior $\mathrm{Beta}(a_0; b_0)$, after s_k clicks and f_k non-clicks the posterior is

$$_k \mid \text{data} \quad \mathrm{Beta}(a_0 + s_k;\ b_0 + f_k); \tag{2}$$

by Beta-Bernoulli conjugacy. At every event TS draws one value $\tilde{}_k$ from each posterior and shows $\arg\max_k \tilde{}_k$; after a matched event it adds one to s_k or f_k (?). Items with an uncertain posterior sometimes draw a high value and get tried, so exploration comes from the posterior itself. The prior can be read as a_0 1 imagined clicks and b_0 1 imagined non-clicks with mean $a_0{=}(a_0 + b_0)$. The default $\mathrm{Beta}(1; 1)$ is flat over $[0; 1]$.

3.3 Upper Confidence Bound

UCB1 (?) first shows every item once and then shows the item with the highest index

$$U_k = \hat{}_k + c\sqrt{\frac{2\ln n}{n_k}}; \tag{3}$$

where $\hat{}_k$ is the item's CTR so far, n_k the number of times it was shown and n the total number of matched events. The second term is an optimism bonus that shrinks as an item is shown more. The textbook version has $c = 1$; we treat c as a tuning parameter. Unlike TS, UCB is deterministic: the same history always gives the same choice.

3.4 Measures

Besides the CTR we report the lift over Random, $100(\widehat{\mathrm{CTR}}{=}\widehat{\mathrm{CTR}}_{\mathrm{Random}} \;\; 1)$, the regret per 10,000 matched events, $10^4(\widehat{\mathrm{CTR}}_{\mathrm{Oracle}} \;\; \widehat{\mathrm{CTR}})$, which counts the clicks lost against always showing the best item, and a two-sided two-proportion z-test against Random based on the clicks and matched events of one run.

4 Question (a): comparing the policies

4.1 Main comparison

Table ?? compares the policies on both files. The defaults are TS with a flat prior and UCB with $c = 1$. The tuned versions use the settings found in question (c): a prior $\mathrm{Beta}(1; 300)$, whose mean of 0.33% is close to the overall CTR, and $c = 0{:}01$.

Table 1: Replay CTR of the policies, 10 runs each (code Table 13). Lift is relative to Random, regret is the number of clicks lost against the Oracle per 10,000 matched events, p is the two-proportion z-test against Random. Tuned settings come from question (c) and were chosen on the same log.

	80 items					10 items		
Policy	Matched	CTR (%)	Lift (%)	Regret	p	CTR (%)	Lift (%)	p
Random (logging policy)	17,192	0.330	0.0	45.4		0.261	0.0	
TS, Beta(1; 1)	17,109	0.377	14.1	40.7	0.47	0.380	45.4	0.05
TS, Beta(1; 300)	17,077	0.413	25.0	37.1	0.21	0.395	51.0	0.03
UCB, c = 1	10,757	0.408	23.4	37.6	0.30	0.330	26.3	0.43
UCB, c = 0:01	11,169	0.502	52.0	28.2	0.02	0.398	52.3	0.03
Per group TS (Section ??)	17,073	0.443	34.2	34.1	0.09			
Oracle (best item)	16,325	0.784	137.3	0.0		0.525	100.9	

Random replays at 0.330% on the 80 item file, close to the 0.348% of the full log, and the Oracle replays at exactly the CTR of the best item. Both checks confirm that the evaluator works (code Table 7).

The two files tell different stories. With 10 items every learning policy beats Random by 45 to 52%, and TS shows the best item in 31.5% of its matched events against 10% under Random (code Table 9). With 79 items the same 17,000 matched events are spread over eight times as many arms, about 217 per item and on average less than one click each. Default TS then barely moves away from uniform: it shows the best item 1.7% of the time against 1.27% for Random, and its 14% lift is not significant ($p = 0.47$). The flat prior is the reason. After n non-clicks its posterior mean is $1/(n + 2)$, so after 217 shows an item without clicks still looks like 0.46%, above the true CTR of most items, and TS keeps exploring until the data runs out. Figure ?? shows the result: the TS curve separates from Random early and then stays flat, far below the Oracle.

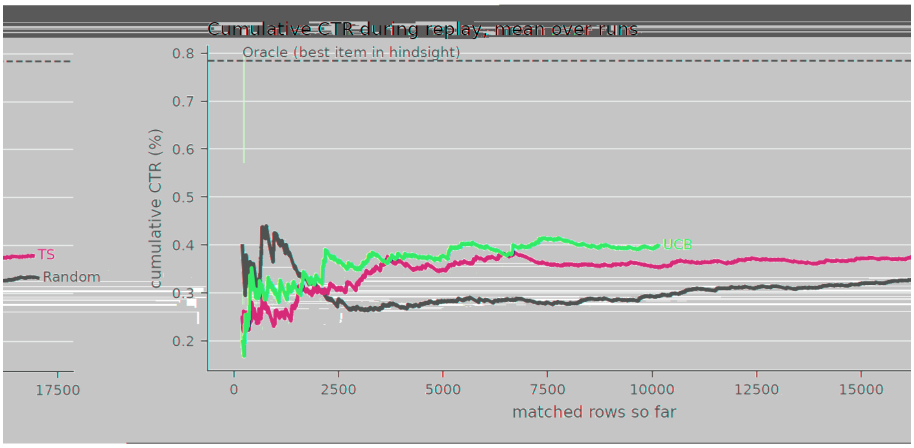

Figure 2: Cumulative CTR during replay on the 80 item file, mean over runs, for TS, UCB ($c = 1$) and Random; the dashed line is the Oracle (code Figure 3).

Tuning changes the ranking. The informed prior lifts TS to 0.413%, and UCB with $c = 0.01$ reaches 0.502%, a 52% lift and the only significant result on the 80 item file ($p = 0.02$). With $c = 1$ the bonus in (??) is about 0.2 after a few thousand events, around 70 times the size of the CTRs it is added to, so UCB mostly rotates through the items; on the 10 item file it shows every item about 10% of the time, exactly like Random (code Table 9). Even the best policy leaves about 60% of the Oracle's advantage over Random on the table: there are simply too few clicks to identify the best of 79 items within one week of uniformly logged data.

4.2 Where to place the items

The brief also asks where to place the items. If the best item depended on the slot, a policy should learn a separate ranking for the left, centre and right position. The five best items per position overlap heavily (items 33, 34, 38, 57 and 59 appear in several slots, code Table 10). A likelihood ratio test of the logistic model with item and position effects against a model with a separate rate for every item and position rejects at the textbook level ($LR = 332.0$, 156 degrees of freedom, $p < 10^{14}$). That test, however, assumes independent rows. When we swap the three position labels between slots at random within every hour, which keeps the streaks intact but removes any real link between slot and item, the placebo statistics average 323.3 and the permutation p-value is 0.38 (code Table 11). We find no evidence that the best item depends on the slot. Consistently, TS with one model per position does not beat one model for all positions (0.399% and 0.395% against 0.387%, code Table 12). The practical answer is to show the best items and not to spend data on learning a ranking per slot.

5 Question (b): batch size

A production system does not update after every impression; it collects results and updates in batches. With batch size B the policy keeps its beliefs fixed for B matched events and then processes all of them at once. We vary B from 1 to 5,000 for TS and for UCB with $c = 0{:}01$ (Table ??).

Table 2: CTR (%) for different batch sizes, 5 runs each (code Table 14). Random replays at 0.261% (10 items) and 0.330% (80 items).

Batch size	1	10	100	500	1,000	5,000
TS, 10 items	0.405	0.392	0.343	0.337	0.365	0.353
UCB $c = 0{:}01$, 10 items	0.425	0.403	0.421	0.348	0.368	0.274
TS, 80 items	0.387	0.353	0.370	0.316	0.356	0.366
UCB $c = 0{:}01$, 80 items	0.569	0.376	0.471	0.465	0.333	0.366

On the 10 item file both policies lose clicks as batches grow, but in different ways. TS loses some of its advantage and then levels off; at $B = 5{;}000$, which leaves only three or four updates in the whole week, it still gets 0.353%, well above Random. UCB hardly suffers up to $B = 100$ and then falls to 0.274%, Random level. The difference follows from the algorithms. During a batch UCB's index does not change, so it shows the same item for the whole batch; a weak choice is repeated thousands of times. TS keeps drawing from its posterior and spreads the batch over several promising items. This agrees with ?, who found that TS copes well with delayed feedback. On the 80 item file the picture is weaker: TS barely learns at any batch size, so there is little to lose, and UCB's results vary so much between runs (standard deviation across runs up to 0.13 percentage points) that the drop is not clear beyond $B = 1$. A second effect shows in the matched counts: while UCB holds one choice during a batch it also catches the streaks in the log, and its matched events rise from 12,930 at $B = 1$ to about 17,000 at B 100 (10 items).

6 Question (c): tuning

We vary the prior of TS and the bonus scale c of UCB (Figure ??).

figure_5_tuning_80items.png not found: run main.py

Figure 3: CTR for TS with prior Beta(1; b) (left) and UCB with bonus scale c (right) on the 80 item file, 5 runs per setting, with 1:96 binomial standard errors; the dashed line is Random (code Figure 5, code Tables 15 and 16).

For TS the prior matters most when there are many arms. On the 80 item file CTR rises from 0.387% with the flat prior to 0.420% with Beta(1; 300) and 0.483% with Beta(1; 1000), whose prior means of 0.33% and 0.10% are on the scale of the real CTRs. A flat prior claims that a CTR of 50% is as likely as one of 0.3%, and the data spends most of its clicks correcting that claim. On the 10 item file, with about 1,700 matched events per item, the data overrides any prior and all settings lie within noise of each other (0.31 to 0.42%, code Table 15).

UCB is more sensitive. On the 10 item file CTR follows a hill in c: 0.300% at $c = 1$, 0.444% at $c = 0.03$ and 0.341% at $c = 0.001$. A large bonus keeps UCB exploring; a very small one makes it commit to whichever item looked best early, which is sometimes an item that was lucky. On the 80 item file the best values are $c = 0.01$ (0.569%) and $c = 0.1$ (0.528%) (code Table 16). A bonus on the scale of the CTRs, not on the scale of $[0, 1]$ rewards for which UCB1 was designed, is what makes UCB work here.

These best settings were chosen on the same log on which they are scored, so they flatter the tuned policies. As a fairer check we choose the setting on days 1 and 2 only and score it on days 3 to 7 (code Table 17). On the 80 item file the choice for UCB is $c = 0.1$, which scores 0.502% on the later days against 0.424% for the default and 0.378% for Random, so tuning UCB helps out of sample. The choice for TS, Beta$(1, 30)$, does not help (0.372% against 0.381%). On the 10 item file the first two days leave only about a dozen matched clicks, too few to separate the settings, and the defaults are chosen. In practice c and the prior have to be set before launch, ideally from earlier data on the site's typical CTR.

Finally, replaying the log in shuffled order removes the streaks. UCB with $c = 1$ then matches 17,112 events instead of 10,639 on the 80 item file (code Table 18). The CTRs are lower in shuffled order for five of the six policy and file combinations, by 0.05 to 0.11 percentage points (for example 0.569% against 0.501% for UCB with $c = 0.01$). Each gap is about one standard error, but the consistent direction suggests that replay on this log slightly favours policies that stay with one item, perhaps because clicks cluster within streaks. [TODO: Random in time order vs shuffled from code Table 18: if Random shows no gap, write that the gap appears only for adaptive policies.] The ranking of the policies does not change, but absolute CTRs should be read with this caveat.

7 Question (d): one model or one per user group

So far every visitor gets the same recommendation. If user groups prefer different items, one model per group could do better, but each group model then learns from a fraction of the data. We compare three designs for each of the four user features: one pooled TS, one TS per level of the feature starting from Beta$(1, 1)$, and one TS per level that borrows from the pooled model. In the last design the prior of item k for group g is

$$a_{0,k} = 1 + w_k p_k, \qquad b_{0,k} = 1 + w_k (1 - p_k), \qquad w_k = \min(m, N_k), \tag{4}$$

where p_k and N_k are the CTR and the number of matched events of item k in the pooled model so far and $m = 1{,}000$. A small group stays close to the pooled estimate; a large group soon outweighs the borrowed m shows with its own data. This is shrinkage towards the pooled mean. The pooled counts are built during the replay from past events only, so the policy uses no information that a live system would not have. A group's own clicks are counted in both terms, which double counts them slightly; the cap m keeps this small.

Table 3: CTR (%) of pooled and per group TS on the 80 item file, 5 runs each (code Table 19). Random gives 0.341% and pooled TS 0.387% in the same runs; one standard error is about 0.05.

	user_feature_0	user_feature_1	user_feature_2	user_feature_3
Levels	4	6	10	10
Per group TS, from zero	0.342	0.374	0.357	0.405
Per group TS, borrowing	0.494	0.388	0.362	0.395

Splitting from zero never beats the pooled model (Table ??). Each group model starts with no knowledge and has too few clicks to learn. Borrowing helps clearly only for user_feature_0, the feature with the fewest and largest groups: 0.494% against 0.387% for the pooled model. For the features with ten levels, many of them small, it makes no difference. Within user_feature_0 all three evaluable groups gain (code Table 20). Group 3, with 150,691 rows, is the clearest case: the pooled model lifts its CTR by 14% over Random, the per group model with borrowing by 100%, so this group prefers different items from the majority. Group 0 has 6,641 rows and about 80 matched events per run, too few to learn anything or even to measure how well it is served. Aggregation wastes the differences between groups; full disaggregation wastes data. Borrowing gets most of both, but only where groups are large enough.

8 Question (e): fairness

A policy that maximises clicks can be unfair in two ways. It can concentrate exposure on a few items, so that the other sellers are hardly seen and an item that was unlucky early never gets a second chance (fairness to items). It can also serve some user groups much better than others, usually small groups or groups whose taste differs from the majority (fairness to users).

We measure fairness to items with the Gini coefficient of the exposure shares e_k (the share of matched events in which item k was shown),

$$G = \frac{2\sum_{i=1}^{K} i\, e_{(i)}}{K \sum_{i} e_{(i)}} \qquad \frac{K+1}{K}; \tag{5}$$

with $e_{(1)} \quad e_{(K)}$, so $G = 0$ means equal exposure. We also count the items that receive less than half of the fair share $1=K$, and report the rank correlation between exposure and item CTR, which shows whether exposure follows merit. For fairness to users we compute, per level of user_feature_0, the CTR divided by Random's CTR in the same group (the lift), and report the worst group's lift and the gap between the best and the worst group. Groups with fewer than 500 matched events per run are left out because their lift cannot be estimated; on the 80 item file this excludes group 0.

As fairness extension we add a minimum exposure guarantee. A wrapper around any policy promises every item at least a fraction of its fair share: whenever some item has been shown fewer than $n=K$ times after n matched events, the wrapper shows the most neglected item; otherwise the base policy decides. We put the floor on UCB, the policy that concentrates exposure, with $= 0{:}25$ and $= 0{:}5$.

Table 4: Clicks against fairness on the 80 item file, 5 runs each (code Table 21). Fair share per item is 1=79 = 1:27%.

		Fairness to items			Fairness to users	
Policy	CTR (%)	Gini	Items < $\frac{1}{2}$ share	Exposure vs CTR	Worst lift	Gap
Random	0.34	0.021	0	0:06	1.00	0.00
TS	0.39	0.090	0	0.69	1.10	0.05
UCB c = 0:01	0.57	0.673	54	0.24	0.92	1.06
UCB + floor = 0:25	[TODO:]					
UCB + floor = 0:5	[TODO:]					
Per group TS, borrowing	0.49	0.157	0	0.67	1.28	0.72

The policies sit at different points of the trade-off (Table ??, Figure ??). UCB with c = 0:01 earns the most clicks, but it puts 41% of all exposure on three items and leaves 54 of the 79 items below half their fair share. It is also the least fair to users: it lifts group 2 by 97% and leaves group 3 slightly below Random (lift 0.92, code Table 22), because its few favourite items suit the majority. TS is close to fair on both counts: its random draws spread exposure (G = 0:09), exposure follows merit (rank correlation 0.69), and every group gains 10 to 15%. The price is a lower CTR. Per group TS with borrowing gives the best deal for users, the highest worst-group lift (1.28) with a moderate Gini of 0.16, because it serves each large group with its own favourites.

[TODO: Floor results from code Table 21 and 22: how much the Gini and the number of starved items fall for = 0:25 and = 0:5, what it costs in CTR, and how the group lifts change. In the 20 item test the floor = 0:5 cut the Gini of UCB from 0.64 to 0.38 and left no item below half its share.]

figure_6_fairness_80items.png not found: run main.py

Figure 4: CTR against the Gini of item exposure on the 80 item file; error bars are 1:96 binomial standard errors (code Figure 6).

9 Question (f): is the data stationary?

Two kinds of change matter differently for a bandit. If all items become more or less popular together, the best item stays the best and a stationary policy still picks correctly. If the ranking of the items changes, a policy that averages over the whole past keeps showing yesterday's winner. We test both on the 80 item file.

All textbook tests assume independent rows, and the streaks in the log violate that. We therefore calibrate every test with a block permutation placebo: the 168 hours of the week are dealt out at random into placebo days (or halves), keeping each hour intact, and the test statistic is recomputed 200 times. A placebo split keeps the dependence between nearby rows but contains no real trend, so it shows how large the statistic gets by chance. The permutation p-value is the share of placebo statistics at least as extreme as the real one.

Table 5: Stationarity tests on the 80 item file (code Tables 23 to 25). Placebo values come from 200 hour block permutations.

Test	Statistic	Textbook p	Placebo mean	Placebo 95%	Perm. p
Equal daily CTR, ^{2} (6 df)	27.5	0.0001	12.4	24.4	0.015
Item ranking, halves (Spearman)	0.772		0.747	0.684[a]	0.74
Item half, LR (78 df)	136.8	0.00004	166.4	207.1	0.89
Item day, LR (468 df)	928.7	< 10^{6}	972.4	1,057.8	0.82

[a] 5th percentile, since a change in the ranking lowers .

The textbook tests all reject stationarity, but the placebo values show why they cannot be trusted here: even random groupings of hours give a ^{2} of 12.4 on average instead of the 6 expected under independence (Table ??). After calibration the results split. The overall daily CTR does change: the real ^{2} of 27.5 beats 98.5% of the placebo values (permutation p = 0:015), with Wednesday at 0.303% and Tuesday at 0.382%. The item ranking does not. The rank correlation of item CTRs between days 1 to 3 and days 4 to 7 is 0.772, a typical value for a placebo split of hours (mean 0.747, p = 0:74; Figure ??). The likelihood ratio tests of an item by period interaction in a logistic model of the click give statistics below the placebo mean